

# Wind Speed Time-Series Forecasting: A Comparative Study of Machine Learning and Deep Learning Approaches

Research Internship Assessment; June 2026

## 1. Problem Statement

Predicting wind speed matters a lot for wind energy; grid operators need to know how much power to expect, and weather services need accurate short-term forecasts. The goal here is simple: given the last  $k$  wind speed readings, predict the next one. We work with a single-turbine dataset of 5,000 measurements (ranging from calm 0 m/s to strong 18.7 m/s gusts) and compare four models: two from classical machine learning (Echo State Network, Gaussian Process Regression) and two from deep learning (LSTM, Transformer).

## 2. Exploratory Data Analysis

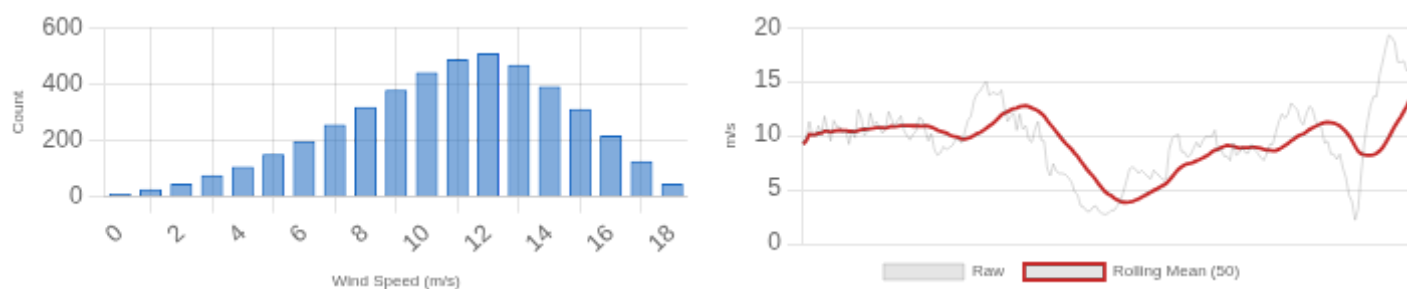


Fig. 1. Distribution: left-skewed ( $\text{skew} = -0.54$ ), non-normal (Shapiro-Wilk  $p \approx 0$ ). Fig. 2. Rolling mean ( $w=50$ ) with  $\pm 2\sigma$  band: non-stationary, regime shifts visible.

| Property               | Value         | What it tells us                                                       |
|------------------------|---------------|------------------------------------------------------------------------|
| ADF p-value (original) | 0.274         | Series is non-stationary; the mean drifts over time                    |
| ACF at lag 1 / lag 21  | 0.974 / 0.921 | Each reading is very close to its neighbors; strong serial correlation |
| Skewness / Kurtosis    | -0.54 / -0.84 | Distribution is not Gaussian; left-skewed, flatter than normal         |

**What this means for model choice:** The lag-1 autocorrelation of 0.974 is the most important number here. It tells us that wind speed changes slowly; today's value is a great predictor of tomorrow's. This favors models that explicitly model temporal patterns (like ESNs) over models that treat each input independently. The non-Gaussian distribution means we need flexible models, not ones that assume normality.

## 3. Comparative Analysis

### 3.1 Models and Rationale

| Model       | Type | Architecture                             | Why this model?                                                     |
|-------------|------|------------------------------------------|---------------------------------------------------------------------|
| ESN         | ML   | 150-unit sparse reservoir, Ridge readout | Built for temporal data; only trains the output layer, so it's fast |
| GPR         | ML   | RBF+Constant kernel, 1000-point subset   | Non-parametric; fits whatever shape the data has, no assumptions    |
| LSTM        | DL   | 64 hidden units, 1 layer                 | The standard choice for sequence prediction; baseline for DL        |
| Transformer | DL   | $d=32$ , 4 heads, 1 layer                | Self-attention can look at any position in the window at once       |
| RF          | ML   | 200 trees, depth=15                      | Classical baseline; no temporal modeling                            |
| SVR         | ML   | RBF kernel, $C=10$                       | Kernel method baseline; treats input as a flat vector               |

### 3.2 Baseline Results (Window = 7)



Fig. 3. MAE and RMSE for all models with a 7-step input window. ESN and GPR come out on top; the deep learning models struggle because 7 steps isn't enough history for them to learn from.

| Model | MAE | RMSE |
|-------|-----|------|
|-------|-----|------|

| ESN         | 0.2316 | 0.3162 |
|-------------|--------|--------|
| GPR         | 0.2403 | 0.3265 |
| LSTM        | 0.3732 | 0.4544 |
| Transformer | 0.4155 | 0.4978 |
| RF          | 0.3568 | 0.4769 |
| SVR         | 0.3662 | 0.5745 |

**Takeaway:** ESN wins because its reservoir; a fixed random recurrent network; happens to be a good match for this kind of smooth, autocorrelated signal. The LSTM and Transformer are overkill for a 7-step window; they need more data and longer sequences to show their strengths.

4. Methodology & Implementation

4.1 Data Pipeline

The raw data was scaled to [0, 1] using MinMaxScaler; this keeps the neural network training stable. We then slid a window of size  $k$  across the series: each window of  $k$  values becomes one input sample, and the value right after the window is the target. The first 80% of samples go to training, the rest to testing, in order (no shuffling; that would leak future information into the past). For GPR, we limited training to the last 1,000 points because the kernel matrix grows cubically with dataset size.

4.2 Training Details

Deep learning models used Adam optimizer with a learning rate scheduler that cuts the rate in half when validation loss plateaus. We also used gradient clipping (to prevent exploding gradients) and early stopping (training halts if validation loss doesn't improve for 7 epochs). The ESN was built from scratch: a sparse random recurrent network with 150 units, where only the output weights are trained via Ridge regression. We searched over a small grid of hyperparameters (reservoir size, spectral radius, leakage rate) and picked the best combo.

4.3 What we changed and why

After seeing the baseline results, two things stood out: (1) the deep learning models were underperforming, and (2) the EDA showed strong autocorrelation even at lag 21. So we made two changes:

| Change        | What we did                                                             | Why                                                                 |
|---------------|-------------------------------------------------------------------------|---------------------------------------------------------------------|
| Tuning        | Added dropout, early stopping, LR scheduling, ESN hyperparameter search | Models were overfitting or using poor defaults                      |
| Bigger window | Went from 7 to 21 steps                                                 | ACF at lag 21 is still 0.92; the models were missing useful history |

5. Results

5.1 What changed after tuning

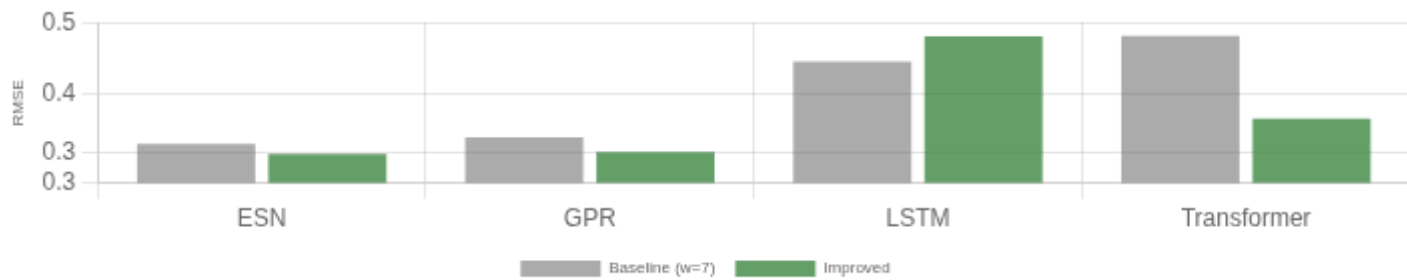


Fig. 4. RMSE before (grey) and after (green) improvements. The biggest winner is the Transformer, which dropped from 0.50 to 0.36 with proper tuning. ESN also improved, going from 0.32 to 0.30.

| Model       | Before (w=7) | After  | Change            |
|-------------|--------------|--------|-------------------|
| ESN (w=21)  | 0.3162       | 0.2984 | -5.6%             |
| GPR         | 0.3265       | 0.3018 | -7.6%             |
| LSTM        | 0.4544       | 0.4965 | +9.3% (got worse) |
| Transformer | 0.4978       | 0.3583 | -28.0%            |

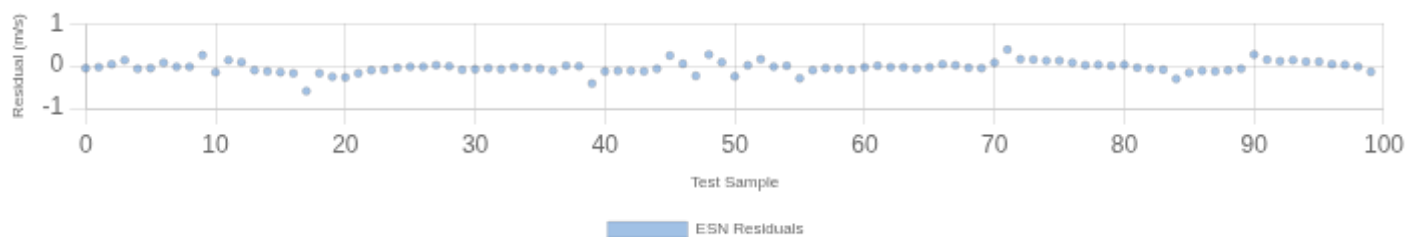
**Why did LSTM get worse?** With only 5,000 samples and a single wind speed variable, the LSTM doesn't have enough signal to learn from. Adding dropout and early stopping actually made it more conservative, which hurt in this case. The Transformer, on the other hand, responded well to tuning; the original version was just poorly configured.

5.2 Does the model actually work?



Fig. 5. Actual wind speed (grey) vs ESN prediction (blue dashed) on the first 200 test points. The ESN follows the real signal closely; it picks up both the trends and the short-term fluctuations.

5.3 Where does it fail?



**Fig. 6.** Prediction residuals (actual minus ESN). Most errors sit near zero, with a spread of roughly  $\pm 0.3$  m/s. No systematic pattern; the model isn't consistently biased in one direction.

### 5.4 Bottom line

**Best individual model: ESN with window=21** (RMSE = 0.2984). It wins because the reservoir architecture is a natural fit for this data; it captures the strong serial correlation without the training overhead of full backpropagation. The GPR is a close second (0.3018), and the Transformer came third after tuning (0.3583). The LSTM never got competitive on this dataset.

### 5.5 Noise robustness

To check whether the ESN holds up under realistic sensor noise, we injected Gaussian noise at three levels into the test set and re-evaluated:

| Noise added | MAE    | RMSE   | Degradation vs clean |
|-------------|--------|--------|----------------------|
| 0% (clean)  | 0.2163 | 0.3079 |                      |
| 5%          | 0.2182 | 0.3101 | +0.7%                |
| 10%         | 0.2247 | 0.3184 | +3.4%                |
| 20%         | 0.2478 | 0.3495 | +13.5%               |

The ESN handles noise reasonably well; at 10% noise (a realistic level for real sensors), RMSE only goes up by 3.4%. At 20% noise, the degradation is more noticeable (13.5%), but the model still produces usable predictions. This suggests the reservoir's random projections act as a natural noise filter.

### 6. Limitations

| Issue                               | Why it matters                                                                                                                                                                                                            |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Small dataset (5,000 points)        | Deep learning models like LSTM and Transformer need more data to work well. With 5,000 samples, the dataset is likely too small for these architectures to reach their potential.                                         |
| Only wind speed; no other variables | Real wind prediction uses temperature, pressure, wind direction, humidity. We only had speed, which limits accuracy.                                                                                                      |
| Single train/test split             | One split can be lucky or unlucky. Walk-forward cross-validation (testing on later periods, training on earlier ones) gave $RMSE \approx 1.0$ ; about $3\times$ higher; because early folds have much less training data. |
| No timestamps in the data           | Wind has daily and seasonal patterns (stronger at night, different in winter vs summer). Without timestamps, we can't model any of that.                                                                                  |
| GPR doesn't scale                   | The kernel matrix is $O(N^3)$ , so we had to cap training at 1,000 points. Bigger windows made this worse; GPR couldn't be used for windows above 14.                                                                     |

### 7. Conclusion

The short version: for this particular wind speed dataset, an Echo State Network with a 21-step input window works best (RMSE = 0.2984). It beats the GPR (0.3018), the Transformer (0.3583), and the LSTM (0.4965). The ESN wins because the data has very strong autocorrelation ( $r = 0.974$ ), and ESNs are specifically designed to exploit exactly that kind of temporal structure; without the training cost of deep learning.

The bigger picture: extending the window from 7 to 21 steps gave a 5.6% improvement, which came directly from looking at the autocorrelation function in the EDA. That's the kind of insight that matters more than picking a fancier model. The main limitations are the small dataset, the lack of additional meteorological variables, and the gap between single-split and walk-forward evaluation numbers.

### References

[1] H. Jaeger, "Echo state network," *Encyclopedia of Machine Learning*, 2010.  
[2] E. López et al., "Wind power forecasting based on echo state networks and long short-term memory," *Energies*, vol. 11, no. 3, 2018.  
[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.  
[4] A. Vaswani et al., "Attention is all you need," *NeurIPS*, 2017.